CO5 – AT1

Constraint-Based Problem Solving

REPORT:

Exam Timetabling using Graph Coloring and Backtracking/CSP

2. Aim

To develop an automated university examination timetable using graph coloring and backtracking techniques such that no student has two examinations at the same time and available rooms satisfy the required capacity constraints.

3. Problem Statement

University examination scheduling is a complex constraint satisfaction problem. Multiple examinations must be assigned to limited time slots and rooms while avoiding conflicts between students.

The main objective is to generate a conflict-free timetable while efficiently utilizing the available time slots and examination rooms.

4. Objectives

- Avoid overlapping examinations for students.
- Assign examinations to available time slots.
- Allocate suitable rooms according to student strength.
- Use graph coloring to represent examination conflicts.
- Use backtracking to find a valid timetable.
- Verify that the generated timetable has no conflicts.

5. Graph Coloring Model

The problem can be represented using an undirected graph.

- **Vertex:** Represents an examination.
- **Edge:** Represents a conflict between two examinations.
- **Color:** Represents an examination time slot.
- **Adjacent vertices:** Cannot have the same color.

For example:

Maths -------- Physics

| |

| |

DBMS ---------- OS

| |

Python -------- Networks

If Maths and Physics have common students, an edge is created between them. Therefore, they cannot receive the same time slot.

Graph coloring is commonly used for university timetabling because exams/courses can be represented as vertices and common-student relationships as edges.

6. Constraints

Hard Constraints

These constraints must always be satisfied.

1. **Student Conflict Constraint**
    - A student cannot write two exams at the same time.
2. **Time Slot Constraint**
    - Every examination must receive one available time slot.
3. **Room Capacity Constraint**

- The number of students taking an examination must not exceed the capacity of its assigned room.

4. **Room Availability Constraint**
    - One room cannot be assigned to two examinations in the same time slot.

Soft Constraints

These constraints improve the quality of the timetable.

- Avoid consecutive examinations for students.
- Distribute examinations evenly.
- Prefer suitable examination periods.
- Minimize the total number of time slots.

7. Constraint Priority

Priority	Constraint	Type
1	No student examination overlap	Hard
2	Room capacity	Hard
3	Room availability	Hard
4	Valid time slot	Hard
5	Avoid consecutive exams	Soft
6	Minimize number of slots	Soft

The hard constraints are checked first because violating them makes the timetable invalid.

8. Algorithm Used

The proposed solution uses **backtracking graph coloring**.

Steps

1. Create a vertex for every examination.
2. Create an edge between exams having common students.
3. Create available time slots as colors.
4. Select an unassigned examination.
5. Try assigning an available time slot.
6. Check all neighboring examinations.
7. If there is no conflict, continue.
8. If a conflict occurs, try another time slot.
9. If no slot works, backtrack to the previous examination.
10. Continue until all examinations are scheduled.

Backtracking is useful because it can systematically explore alternatives when a partial timetable leads to an invalid assignment. Research on university timetabling also describes graph coloring combined with backtracking and constraint handling as a practical approach.

9. Pseudocode

START

Read examinations

Read student conflicts

Create conflict graph

Read available time slots

For each examination

Try each available time slot

If no neighboring exam uses that slot:

Assign the slot

Recursively schedule next exam

If successful:

return TRUE

Remove the assignment

If all exams are assigned:

Return TRUE

Otherwise:

Return FALSE

Allocate rooms according to capacity

Verify all constraints

Display timetable

STOP

10. Backtracking Logic

Suppose three time slots are available:

Slot 1 = 9:00 - 11:00

Slot 2 = 11:30 - 1:30

Slot 3 = 2:00 - 4:00

The algorithm attempts:

Maths → Slot 1

Physics → Slot 2

DBMS → Slot 2

OS → Slot 1

If a later examination cannot be assigned without creating a conflict, the algorithm goes backward:

Previous assignment

↓

Remove assignment

↓

Try another time slot

↓

Continue scheduling

This process continues until a valid assignment is obtained.

11. Room Allocation

After assigning time slots, suitable rooms are selected based on student strength.

For example:

Room	**Capacity**
Room A	60
Room B	45
Room C	35

If an examination has 50 students, Room A can be selected because its capacity is 60.

A room with capacity 35 cannot be used for that examination.

Time Complexity

Graph coloring is computationally difficult in its general form, and exam timetabling is consequently an NP-hard/NP-complete-style combinatorial problem depending on the exact formulation.

For backtracking with n examinations and k available time slots, the worst-case search can be approximately:

$O(k^n)$

Although the worst case is exponential, constraint checking and backtracking can eliminate many invalid possibilities early.

Space Complexity

The space complexity of the backtracking graph coloring algorithm is:

$O(V + E + V) = O(V + E)$

Result

The examination timetable was successfully generated using graph coloring and backtracking.

The generated timetable satisfies the major hard constraints:

- No student conflicts.
- Valid time-slot assignment.
- Suitable room capacity.
- Conflict verification completed successfully.